

Experiment Name: Dot Matrix Animation Using 8086 Microprocessor and 8255 PPI

Objective:

1. To interface an 8×8 dot matrix display with the 8086 microprocessor.
2. To understand the working principle of the 8255 Programmable Peripheral Interface (PPI).
3. To generate LED animation using Assembly Language programming.
4. To observe data transfer between the microprocessor and display device.

Theory:

A Dot Matrix Display consists of LEDs arranged in rows and columns. It is widely used to display characters, symbols, and animations. In this experiment, the 8086 microprocessor controls the dot matrix display through the 8255 Programmable Peripheral Interface (PPI).

The 8255 PPI provides programmable input/output ports for interfacing external devices with the microprocessor. Port A and Port B are used to control the rows and columns of the dot matrix display.

The assembly program continuously rotates a binary pattern using the **ROL** instruction. This rotating pattern creates a moving LED animation effect on the display. A delay routine is used to slow down the execution speed so that the animation becomes visible to the human eye.

Required Components:

- 8086 Microprocessor
- 8255 PPI
- 8×8 Dot Matrix Display
- Connecting Wires
- Power Supply
- Proteus Simulation Software

The screenshot shows the Proteus Professional Schematic Capture window. The title bar indicates the file path: C:\Users\tasni\Downloads\trix_Display\Dot_Matrix_ANIMATION.pdsprj - Proteus Professional - Schematic Capture. The menu bar includes File, View, Edit, Tools, Design, Graph, Debug, Library, Template, System, and Help. The toolbar contains various icons for schematic capture, including a Schematic Capture icon. The left sidebar shows the 2D Graphics Windows Mode and Markers - ORIGIN. The main workspace displays a circuit diagram with three integrated circuits (U1, U2, and U3) and a 7-segment display. U1 and U2 are 74VHC125 octal monostable multivibrators, and U3 is a 74VHC125 octal monostable multivibrator. The circuit is connected to a 5V supply and a 10k resistor. The 7-segment display is connected to the outputs of the ICs. The circuit is labeled 'ROOT - Root sheet 1'.

CODE SEGMENT

PPIC_C EQU 06H

PPIB EQU 02H

PPIB EQU 02H

PPIA EQU 00H

ORG 0000H

MOV AL,10000000B ; Configure 8255: Mode 0, all ports as output

```
OUT PPIC_C,AL    ; Send control word to Control Register
```

MOV BL, 1111110B ; Initialize column scanning pattern in BL

L1:

MOV AL,11111111B ; Set all rows high (turn on row LEDs)

OUT PPIA,AL ; Send row data to Port A

```
MOV AL,BL      ; Move current column pattern to AL
```

OUT PPIB,AL ; Send column data to Port B

CALL DELAY ; Hold display pattern for human visibility

ROL BL,1 ; Rotate bit left to shift active column

JMP L1 ; Infinite loop to keep animation running

DELAY:

MOV CX,0FFFFH ; Load maximum count for delay loop

TIMER1:

NOP ; No Operation (wastes clock cycles)

NOP

NOP

NOP

LOOP TIMER1 ; Decrement CX, jump to TIMER1 if CX != 0

RET ; Return from subroutine

CODE ENDS

END

Procedure:

1. Open Proteus simulation software.
2. Create a new project and place the following components:
 - 8086 Microprocessor
 - 8255 PPI
 - 8×8 Dot Matrix Display
3. Connect the 8086 microprocessor data and control buses with the 8255 PPI properly.
4. Connect the dot matrix display rows to Port A and columns to Port B of the 8255 PPI.
5. Write the Assembly Language program using an assembler (e.g., emu8086).
6. Compile the program and generate the HEX/BIN file.
7. Load the compiled HEX/BIN file into the 8086 microprocessor in the Proteus workspace.
8. Run the simulation.
9. Observe the LED animation pattern on the dot matrix display.
10. Verify that the animation continuously moves due to the rotating binary data.

Code Explanation:

- **MOV AL,10000000B**: Loads the control word byte (80H) into the AL register. This pattern configures the 8255 PPI in Mode 0 with Port A, Port B, and Port C set as outputs.
- **OUT PPIC_C,AL**: Sends the control word from register AL to the 8255 Control Register address (06H) to apply the configuration.
- **MOV BL,11111110B**: Initializes the BL register with a column activation mask, where only the first column (Bit 0) is grounded (active low).

- **OUT PPIA, AL**: Outputs the value from AL (**11111111B**) to Port A (**00H**), pulling all row anodes high to prepare them for illumination.
- **OUT PPIB, AL**: Outputs the column-select mask from AL to Port B (**02H**), grounding a specific column to complete the circuit for that column's LEDs.
- **ROL BL, 1**: Rotates the bits in the BL register left by one position. This shifts the active low bit to the next column, creating the movement effect.
- **CALL DELAY**: Redirects execution to the delay subroutine, pausing the program briefly so the human eye can perceive the active column state.
- **LOOP TIMER1**: Automatically decrements the CX register by 1. If CX is not zero, it jumps back to the **TIMER1** label, maintaining the delay loop.

Observation:

The LEDs on the dot matrix display moved continuously from one side to another, producing a smooth animation effect.

Result:

The dot matrix display successfully displayed a moving LED animation pattern using the 8086 microprocessor and 8255 PPI interfacing.

Discussion:

This experiment demonstrated how the 8086 microprocessor can control external hardware devices through the 8255 PPI. The rotating binary pattern generated animation on the dot matrix display. The delay routine controlled the movement speed effectively. The experiment helped in understanding Assembly Language programming, hardware interfacing, and display control techniques.

Conclusion:

The experiment was completed successfully. The 8086 microprocessor interfaced with the 8255 PPI and controlled the dot matrix display to generate continuous LED animation.

References:

1. Intel 8086 Microprocessor Manual
2. Intel 8255 PPI Datasheet
3. Douglas V. Hall – *Microprocessors and Interfacing*
4. Proteus Simulation Software Documentation